

NavPath Academy -- Django Web Application Technical Documentation

1. Architecture Overview

The NavPath Academy web application is a full-stack Django monolith connected to a Supabase-hosted PostgreSQL database. It serves the same maritime exam-prep product as the Flutter mobile app through a browser-based interface, with a fully functional backend, real database persistence, and a Django Admin panel for content management.

- Framework: Django 5.2.17
 - Language: Python 3.12
 - Architecture Pattern: MVT (Model-View-Template) -- Django's standard pattern.
 - Database: PostgreSQL hosted on Supabase (connection pooling via PgBouncer, port 5432).
 - Static Files: WhiteNoise middleware (gzip-compressed, fingerprinted).
 - Deployment Target: Render (WSGI via Gunicorn).
-

2. Project Structure

```
Web/
  navpath_web/          # Project config package
    settings.py         # All settings (environ-driven)
    urls.py             # Root URL dispatcher
    wsgi.py             # WSGI entry point for Gunicorn

  core/                # Public-facing pages + shared models
  users/               # Authentication, dashboard, profile
  courses/             # Course catalogue, lessons, mock tests, study materials
  enrollments/         # Enrollment logic and checkout

  templates/           # All HTML templates (Jinja2-style Django templating)
    base.html          # Shared nav, footer, Tailwind CDN, Inter font
    core/              # Home, About, Public Support
    courses/           # Course list, categories, course detail
    dashboard/         # Dashboard, player, mock tests, progress, profile, settings ...
    enrollments/       # Checkout page
    users/             # Login, Register

  static/              # Source static assets (CSS, images, logo)
  staticfiles/         # Generated by collectstatic -- served by WhiteNoise
  manage.py
  requirements.txt
  .env                 # Local environment variables (never committed)
```

3. Django Apps

``core``

Public-facing pages that require no login.

Model	Purpose
<code>`Notification`</code>	Per-user notifications with type, read f
<code>`Message`</code>	Internal user-to-user messaging (inbox/o
<code>`UserSettings`</code>	Per-user preferences (email/SMS/push not
<code>`SupportTicket`</code>	User-raised support requests with status
<code>`FAQ`</code>	Publicly visible FAQ entries, ordered an
<code>`CalendarEvent`</code>	User calendar entries (lessons, tests, l

Management Command: `python manage.py seed_data --` populates the database with all course categories, courses, lessons, mock test questions, study materials, and sample FAQs for a fresh install.

``users``

Handles all authentication and the logged-in dashboard shell.

Custom model: `CustomUser` (extends `AbstractUser`) -- adds `phone_number`.

Signal: `post_save` on `CustomUser` auto-creates a `UserProfile`.

URL	View	Auth Required
<code>`/users/login/`</code>	<code>`login_view`</code>	No
<code>`/users/register/`</code>	<code>`register_view`</code>	No
<code>`/users/logout/`</code>	<code>`logout_view`</code>	No
<code>`/users/dashboard/`</code>	<code>`dashboard_view`</code>	[OK]
<code>`/users/progress/`</code>	<code>`progress_view`</code>	[OK]
<code>`/users/profile/`</code>	<code>`profile_view`</code>	[OK]
<code>`/users/notifications/`</code>	<code>`notifications_view`</code>	[OK]
<code>`/users/messages/`</code>	<code>`messages_view`</code>	[OK]
<code>`/users/settings/`</code>	<code>`settings_view`</code>	[OK]
<code>`/users/support/`</code>	<code>`support_view`</code>	[OK]
<code>`/users/raise-ticket/`</code>	<code>`raise_ticket_view`</code>	[OK]
<code>`/users/calendar/`</code>	<code>`calendar_view`</code>	[OK]
<code>`/users/live-classes/`</code>	<code>`live_classes_view`</code>	[OK]

``courses``

All course content, lesson playback, mock tests, and study materials.

Model	Purpose
`Category`	Course grouping with name, emoji icon, a
`Course`	Full course record: title, category, des
`Lesson`	Ordered lessons within a course; stores
`StudyMaterial`	Attached resources (PDF, Note, Video Lin
`MockTestQuestion`	4-option MCQ per course; stores correct
`LessonCompletion`	Per-user, per-lesson completion record (
`MockTestResult`	Stores score, total, and timestamp per t

URL	View	Auth Required
`/courses/`	`course_list`	No
`/courses/categories/`	`course_categories`	No
`/courses/<pk>/`	`course_detail`	No
`/courses/my-courses/`	`my_courses_view`	[OK]
`/courses/player/`	`course_player_view`	[OK]
`/courses/mark-complete/`	`mark_lesson_complete` (POST)	[OK]
`/courses/mock-tests/`	`mock_tests_view`	[OK]
`/courses/mock-tests/submit/`	`submit_mock_test` (POST)	[OK]
`/courses/test-results/`	`test_results_view`	[OK]
`/courses/study-materials/`	`study_materials_view`	[OK]

`enrollments`

Handles the enroll action and checkout page.

Model	Purpose
`Enrollment`	Links a user to a course; tracks `progre

update_progress() -- recalculates progress by counting LessonCompletion records vs. total lessons; called automatically on dashboard load and after each lesson completion.

URL	View	Auth Required
`/enrollments/checkout/`	`checkout_view`	No
`/enrollments/enroll/`	`enroll_action` (POST)	[OK]

4. Feature Documentation & Status

- [Implemented]: Fully functional with real database persistence.
- [Partial]: Feature works but has UI placeholder elements.

- [Planned]: Not yet implemented.

Authentication

- Login [Implemented] -- Django AuthenticationForm; session-based; redirects to dashboard on success.
- Registration [Implemented] -- Creates CustomUser + auto-creates UserSettings; immediately logs the user in.
- Logout [Implemented] -- Clears session; redirects to home.
- Password Reset [Planned] -- Django has built-in views; not yet wired up.

Course Browsing

- Home Page [Implemented] -- Displays all categories fetched from the database.
- Course List [Implemented] -- Filterable by category slug and searchable by title.
- Course Detail [Implemented] -- Lessons, study materials, enrolment status, and completion data all from DB.
- Categories Page [Implemented] -- All categories with course counts.

Enrollment & Checkout

- Enroll Action [Implemented] -- get_or_create on Enrollment; idempotent; real DB record.
- Checkout Page [Partial] -- UI exists; no payment gateway integrated.
- Payments [Planned] -- Stripe / Razorpay integration.

Learning Experience

- Course Player [Implemented] -- Loads lesson by ?lesson=<id>&course=<id>; marks completions; tracks next lesson.
- Mark Lesson Complete [Implemented] -- POST endpoint; creates LessonCompletion; recalculates progress; returns JSON for AJAX.
- Progress Tracking [Implemented] -- Real-time progress_percentage on Enrollment; shown on dashboard and My Courses.
- Mock Tests [Implemented] -- Loads all MockTestQuestion for an enrolled course; scores and saves MockTestResult.
- Test Results [Implemented] -- Session-based result display on submit; history from DB on subsequent visits.
- Study Materials [Implemented] -- Lists StudyMaterial records per enrolled course; file URLs served as links.
- Video Playback [Partial] -- Video URL field stored in Lesson; player renders an HTML <video> or iframe from the URL; no streaming backend.
- Live Classes [Planned] -- Page scaffolded; no video conferencing integration.
- Calendar [Planned] -- Page scaffolded; CalendarEvent model exists; UI not yet wired to DB events.

User Account

- Dashboard [Implemented] -- Shows overall progress, lessons completed, test count, avg score, and continue-learning widget; all from DB.
- Profile Edit [Implemented] -- Updates first_name, last_name, email on CustomUser; persists.
- Notifications [Implemented] -- Reads Notification records for the user; marks all as read on page load.
- Messages / Inbox [Implemented] -- Shows received Message records; auto-creates a welcome message from admin on first visit.
- Settings [Implemented] -- Saves UserSettings (notification prefs, dark mode toggle) to DB.
- Support Tickets [Implemented] -- Creates SupportTicket records; admin can update status via Django

Admin.

- FAQs [Implemented] -- Pulled from FAQ model; admin-managed.

5. Technology Stack

Layer	Technology
Framework	Django 5.2.17
Language	Python 3.12
Database	PostgreSQL (Supabase)
DB Driver	psycopg2-binary 2.9.10
Environment Config	django-environ 0.11.2
WSGI Server	Gunicorn 23.0.0
Static Files	WhiteNoise 6.8.2
CSS Framework	Tailwind CSS (CDN)
Font	Inter (Google Fonts CDN)
Icons	Font Awesome 6.4 (CDN)
Deployment	Render

6. Settings & Environment

All secrets are loaded from .env via django-environ. No secrets are hardcoded.

Variable	Purpose
`SECRET_KEY`	Django cryptographic key
`DEBUG`	`True` locally, `False` on Render
`DATABASE_URL`	Full PostgreSQL connection string (Supab
`ALLOWED_HOSTS`	Comma-separated list of valid hostnames

Production-only security settings (auto-enabled when DEBUG=False):

- SECURE_SSL_REDIRECT = True
 - SECURE_HSTS_SECONDS = 31536000 (1 year, with subdomains + preload)
 - SESSION_COOKIE_SECURE = True
 - CSRF_COOKIE_SECURE = True
 - X_FRAME_OPTIONS = 'DENY'
-

7. Deployment (Render)

Deployment is fully automated via render.yaml at the repo root.

Build command (runs on every deploy):

```
pip install -r requirements.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput
```

Start command:

```
gunicorn navpath_web.wsgi:application --bind 0.0.0.0:$PORT --workers 2 --timeout 120
```

Required Render Dashboard environment variables:

Variable	Value
`SECRET_KEY`	Strong random key
`DATABASE_URL`	Supabase `postgresql://` URL
`ALLOWED_HOSTS`	`your-service.onrender.com`
`DEBUG`	`False` (pre-set in render.yaml)

8. Django Admin

The admin panel at /admin/ is fully configured for all content management.

App	Registered Models	Notable Features
`courses`	Category, Course, Lesson, StudyMaterial	Lessons/Materials/Questions editable in
`core`	Notification, Message, UserSettings, Sup-	
`users`	CustomUser, UserProfile	--
`enrollments`	Enrollment	--

9. Testing

The project includes two test files (Web/check_issues.py and Web/test_mock_e2e.py) for manual/integration checks.

Formal automated unit tests are not yet wired into Django's test runner. Below are the key proposed test cases.

Proposed Test Cases

1. Enrollment is idempotent

- ***Action*:** POST /enrollments/enroll/ twice with the same course_id for the same user.
- ***Expected*:** Second request returns "already enrolled" message; only one Enrollment record exists.

2. Lesson completion updates progress

- ***Action*:** POST /courses/mark-complete/ with lesson_id and course_id.

- ***Expected*:** LessonCompletion created; JSON response contains updated progress integer; Enrollment.progress_percentage updated in DB.

3. Mock test scoring

- ***Action*:** POST /courses/mock-tests/submit/ with all correct answers.
- ***Expected*:** MockTestResult created with score == total; redirect to test results page showing 100%.

4. Static files served in production

- ***Action*:** GET /static/css/style.css with DEBUG=False.
 - ***Expected*:** 200 response served by WhiteNoise (not Django's dev server).
-

10. Self-Audit (Code vs Docs)

- Docs reflect Code: Yes. Every model, view, URL, and feature described above exists in the codebase exactly as documented.
- Mismatches: None found.
- Hallucinations: None. All unimplemented features are explicitly labeled [Partial] or [Planned]. No backend logic is described as functional unless it persists to the Supabase PostgreSQL database.